

LAPORAN PRAKTIKUM PEMOGRAMAN BERORIENTASI OBJEK

“Relasi Antar Objek (Class Diagram & Asosiasi)”

disusun Oleh:

Sofian Arba’i (2511533029)

Dosen Pengampu:

Nurfiah, S.ST, M.Kom.

Asisten Praktikum:

M Dawi Syauqi



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Assalamualaikum Warahmatullahi Wabarakatuh

Puji syukur penulis panjatkan ke hadirat Allah SWT atas rahmat dan karunia-Nya sehingga laporan praktikum Pemrograman Berorientasi Objek Pekan 2 dengan topik Relasi Antar Objek (Class Diagram dan Asosiasi) ini dapat diselesaikan dengan baik. Shalawat serta salam semoga senantiasa tercurahkan kepada Nabi Muhammad SAW.

Laporan ini disusun sebagai bentuk pertanggungjawaban atas pelaksanaan praktikum Pemrograman Berorientasi Objek yang membahas hubungan antar objek, pemodelan class menggunakan class diagram, serta implementasi relasi asosiasi menggunakan bahasa pemrograman Java.

Penyusunan laporan ini tidak terlepas dari bantuan, bimbingan, dan dukungan berbagai pihak. Oleh karena itu, saya ingin mengucapkan terima kasih yang sebesar-besarnya kepada:

1. Dosen pengampu mata kuliah Praktikum Pemrograman Berorientasi Objek yang telah memberikan ilmu, arahan, serta bimbingannya.
2. Asisten praktikum yang telah membimbing dan memberikan penjelasan selama kegiatan praktikum berlangsung.

Saya meminta maaf jika masih terdapat kesalahan pada penulisan laporan ini. Oleh karena itu, saya menerima kritik dan saran yang membangun demi perbaikan di masa mendatang.

Padang, 18 September 2026

Sofian Arba'i

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Tujuan.....	1
1.3 Manfaat.....	2
BAB II PEMBAHASAN.....	3
2.1 Pengertian Class Diagram	3
2.1.1 Relasi Asosiasi (One-to-Many)	3
2.2 Implementasi Relasi Antar Objek (Class Diagram & Asosiasi).....	4
2.2.1 Class Rekening	4
2.2.2 Class Main	7
2.2.3 Class Transaksi	11
BAB III PENUTUP	13
3.1 Kesimpulan.....	13
3.2 Saran	13
DAFTAR PUSTAKA	14

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman berorientasi objek, sebuah program dapat dibangun dari beberapa objek yang saling berhubungan untuk menjalankan fungsi tertentu. Salah satu cara untuk menggambarkan struktur dan hubungan antar objek tersebut adalah menggunakan class diagram. Class diagram merupakan gambaran statis yang menunjukkan kelas, komponen, serta hubungan yang terdapat dalam suatu sistem sehingga dapat membantu dalam proses perancangan program.

Salah satu hubungan antar class yang digunakan dalam pemrograman berorientasi objek adalah asosiasi. Asosiasi menunjukkan bahwa suatu class memiliki hubungan dengan class lainnya dan dapat memiliki bentuk seperti one-to-one, one-to-many, maupun many-to-many. Pada Java, hubungan tersebut dapat diterapkan melalui objek yang dimiliki oleh suatu class.

Pada praktikum pertemuan kedua ini, konsep asosiasi diterapkan pada sistem perbankan mini yang dikembangkan dari praktikum sebelumnya. Program menggunakan tiga class utama, yaitu Rekening, Transaksi, dan Main, dengan menerapkan hubungan one-to-many antara Rekening dan Transaksi. Setiap rekening dapat memiliki banyak transaksi yang disimpan menggunakan ArrayList, sehingga transaksi setor dan tarik tunai dapat dicatat sebagai riwayat transaksi.

1.2 Tujuan

Tujuan yang diharapkan pada praktikum pekan 2 tentang Relasi Antar Objek (Class Diagram & Asosiasi) yaitu sebagai berikut.

1. Memahami konsep class diagram dan relasi antar objek dalam pemrograman berorientasi objek.
2. Memahami penerapan relasi asosiasi one-to-many pada program Java.

3. Mengimplementasikan hubungan antara class Rekening dan Transaksi.
4. Memahami penggunaan ArrayList untuk menyimpan kumpulan objek dalam suatu class.

1.3 Manfaat

Manfaat yang didapatkan setelah melakukan praktikum pekan 2 Relasi Antar Objek (Class Diagram & Asosiasi) yaitu sebagai berikut.

1. Menambah pemahaman mengenai hubungan antar class dalam pemrograman berorientasi objek.
2. Mampu menerapkan relasi asosiasi *one-to-many* ke dalam program Java.
3. Mampu mengelola dan menyimpan beberapa objek menggunakan ArrayList.
4. Meningkatkan kemampuan dalam mengembangkan program Java berdasarkan class diagram.

BAB II

PEMBAHASAN

2.1 Pengertian Class Diagram

Diagram kelas atau class diagram adalah jenis diagram statis yang menunjukkan banyak komponen sistem secara grafis. Umumnya para pengembang menggunakannya untuk menguraikan bagaimana sebuah sistem dapat terlihat dalam bentuk statis. Biasanya, para pengembang atau developer menggunakan kelas diagram dalam menggambarkan bagaimana bentuk statis sebuah sistem ketika menciptakan software.

Diagram ini juga memudahkan para developer untuk menentukan kelas yang dibutuhkan koneksinya pada komponen sistem lainnya serta fungsionalitasnya. Dapat dikatakan bahwa diagram class merupakan cetak biru sistem yang digunakan untuk menunjukkan bagaimana hubungan antar bagian konstituen sistem serta menjelaskan fungsi dan layanan masing-masing.

Class diagram memiliki sifat statis artinya tidak menjelaskan mengenai apa yang terjadi apabila kelas-kelasnya berhubungan, tetapi menjelaskan hubungan apa yang terjadi. Hal ini sangat sesuai apabila diimplementasikan pada proyek yang memakai konsep objek oriented. Alasannya, gambaran dari diagram kelas sangat mudah digunakan. Untuk desain model diagram kelas dibagi menjadi dua. Bagian pertama yaitu penjabaran dari database, sedangkan bagian kedua yaitu bagian modul mvc yang mempunyai class kontrol, class interface, dan class entity.

2.1.1 Relasi Asosiasi (One-to-Many)

Sederhananya, association menunjukkan bahwa sebuah class memiliki hubungan dengan class lain. Hubungan atau relasi ini dapat berbagai macam, seperti one to one, one to many, atau many to many. Dari diagram sebelumnya, kamu dapat melihat bahwa association digambarkan dengan sebuah garis

panah dengan menuliskan jenis association-nya. Pada Java, hubungan tersebut dapat terlihat melalui object yang dimiliki pada sebuah class.

2.2 Implementasi Relasi Antar Objek (Class Diagram & Asosiasi)

Pada praktikum pertemuan kedua ini, program dikembangkan dengan menerapkan konsep relasi antar objek menggunakan asosiasi *one-to-many*. Program terdiri dari tiga class utama, yaitu Transaksi, Rekening, dan Main. Implementasi dari topik Relasi Antar Objek (Class Diagram & Asosiasi) pada pertemuan kedua praktikum pemrograman berorientasi objek yaitu melanjutkan sebuah minibank dari pertemuan sebelumnya dengan membuat objek Transaksi setiap kali terjadi aktivitas penyetoran atau penarikan tunai, lalu menyimpannya di dalam objek Rekening. Berikut implementasinya dalam kode program java.

2.2.1 Class Rekening

Kode program Rekening digunakan untuk mengelola data rekening dan aktivitas setor serta tarik tunai. Class ini juga menyimpan banyak objek Transaksi sebagai riwayat transaksi menggunakan ArrayList. Dengan demikian, class ini menerapkan relasi asosiasi *one-to-many* antara Rekening dan Transaksi. Berikut adalah kode program dari class Rekening:

```

1. package pekan2;
2.
3. import java.util.ArrayList;
4. public class Rekening {
5.     String nomorRekening;
6.     String namaPemilik;
7.     double saldo;
8.
9.     // Implementasi Asosiasi (1-to-many)
10.    ArrayList<Transaksi> riwayatTransaksi;
11.
12.    public Rekening(String nomor, String nama, double
    saldoAwal) {
13.        nomorRekening = nomor;
14.        namaPemilik = nama;
15.        saldo = saldoAwal;
16.    }

```

```

17.         // Wajib menginisialisasi ArrayList di dalam
    constructor agar tidak NullPointerException
18.         this.riwayatTransaksi = new ArrayList<>();
19.
20.         System.out.println("Rekening atas nama " +
    namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo);
21.
22.     }
23.
24.     public String getNomorRekening() {
25.         return nomorRekening;
26.
27.     }
28.
29.     public void setorTunai (double nominal) {
30.         if (nominal > 10000 && nominal < 500000) {
31.             saldo += nominal;
32.             // Merekam riwayat (pembuatan objek
    Transaksi di dalam method)
33.             String idTrx = "TRX-S-" +
    System.currentTimeMillis();
34.             Transaksi trxBaru = new Transaksi (idTrx,
    "Kredit", nominal);
35.             riwayatTransaksi.add(trxBaru);
36.             System.out.println("Setor tunai Rp" +
    nominal + " berhasil. Saldo saat ini: Rp" + saldo);
37.         } else {
38.             System.out.println("Gagal: Nominal setor
    harus lebih dari 0! atau Transaksi lebih dari 500000");
39.         }
40.     }
41.     public void tarikTunai(double nominal) {
42.         if (nominal < 10000) {
43.             System.out.println("Transaksi Gagal: Minimal
    nominal penarikan 10.000");
44.         }
45.         else if (nominal > saldo) {
46.             System.out.println("Transaksi Gagal: Saldo tidak
    mencukupi. Saldo Anda: Rp" + saldo);
47.         }
48.         else {
49.             // Merekam riwayat (pembuatan objek Transaksi di
    dalam method)
50.             String idTrx = "TRX-T-" +
    System.currentTimeMillis();
51.             Transaksi trxBaru = new Transaksi (idTrx, "Debit",
    nominal);
52.             riwayatTransaksi.add(trxBaru);
53.             saldo -= nominal;
54.             System.out.println("Tarik tunai Rp" + nominal + "
    berhasil. Saldo saat ini: Rp" + saldo);
55.         }
56.     }
57.     public void cetakMutasi() {

```

```

58.         if (riwayatTransaksi.isEmpty()) {
59.             System.out.println("Belum ada transaksi pada
rekening ini");
60.         } else {
61.             for (Transaksi t : riwayatTransaksi) {
62.                 t.cetakDetail();
63.             }
64.         }
65.     }
66.
67.     public void cekInformasi() {
68.         System.out.println("-- INFO REKENING --");
69.         System.out.println("No. Rekening : " +
nomorRekening);
70.         System.out.println("Nama Pemilik : " +
namaPemilik);
71.         System.out.println("Saldo Akhir : Rp" + saldo);
72.         System.out.println("-----");
73.     }
74. }

```

Class Rekening digunakan untuk menyimpan informasi rekening dan mengelola transaksi. Class ini memiliki atribut nomorRekening, namaPemilik, dan saldo, serta atribut riwayatTransaksi bertipe ArrayList<Transaksi>. riwayatTransaksi merupakan penerapan asosiasi one-to-many, karena satu objek Rekening dapat memiliki banyak objek Transaksi. ArrayList tersebut diinisialisasi pada constructor agar dapat digunakan untuk menyimpan riwayat transaksi.

Method setorTunai() digunakan untuk menambah saldo dan mencatat transaksi penyetoran sebagai objek Transaksi dengan jenis "Kredit" dan ID berawalan "TRX-S-". Sementara itu, tarikTunai() digunakan untuk mengurangi saldo setelah memastikan nominal penarikan memenuhi ketentuan dan saldo mencukupi. Jika penarikan berhasil, program membuat objek Transaksi dengan jenis "Debit" dan ID berawalan "TRX-T-", kemudian menyimpannya ke dalam riwayatTransaksi. Dengan demikian, transaksi yang berhasil akan otomatis tercatat dalam riwayat.

Selain itu, class Rekening memiliki method cetakMutasi() untuk menampilkan seluruh riwayat transaksi menggunakan perulangan for-each dan method cetakDetail() milik objek Transaksi. Jika belum terdapat transaksi,

program menampilkan pesan bahwa belum ada transaksi. Method `cekInformasi()` digunakan untuk menampilkan nomor rekening, nama pemilik, dan saldo akhir, sedangkan `getNomorRekening()` digunakan untuk mengambil nomor rekening ketika proses pencarian akun dilakukan pada class `Main`.

2.2.2 Class Main

Kode program `Main` digunakan untuk menjalankan dan mengatur seluruh proses pada sistem perbankan mini. Class ini menyediakan menu yang memungkinkan pengguna membuat rekening, melakukan transaksi, melihat informasi, mengganti akun, dan melihat riwayat transaksi. Dengan demikian, `Main` berperan sebagai pengendali interaksi antara pengguna dengan objek `Rekening` dan `Transaksi`. Berikut merupakan kode program dari class `Main`:

```

1. package pekan2;
2. import java.util.Scanner;
3. import java.util.ArrayList;
4.
5. public class Main {
6.
7.     public static void main(String[] args) {
8.         Scanner input = new Scanner (System.in);
9.
10.        ArrayList<Rekening> daftarRekening = new
11.        ArrayList<>();
12.        Rekening akunAktif = null; // objek belum
13.        diinisiasi
14.        boolean isRunning = true;
15.
16.        System.out.println("==== SISTEM PERBANKAN MINI
17.        ===");
18.
19.        while (isRunning) {
20.            System.out.println("\nMenu Utama:");
21.            System.out.println("1. Buka Rekening Baru");
22.            System.out.println("2. Setor Tunai");
23.            System.out.println("3. Tarik Tunai");
24.            System.out.println("4. Cek Informasi
25.            Rekening");
26.            System.out.println("5. Ganti Akun");

```

```

24.         System.out.println("6. Cetak Mutasi
(Riwayat)");
25.         System.out.println("7. Challenge 3");
26.         System.out.println("0. Keluar");
27.         System.out.println("Pilih Menu:");
28.
29.         if (!input.hasNextInt()) {
30.             System.out.println("Error: Pilihan
menu harus berupa angka!");
31.             input.next();
32.             continue;
33.         }
34.
35.         int pilihan = input.nextInt();
36.         input.nextLine(); //Membersihkan Buffer
Enter
37.
38.         switch (pilihan) {
39.             case 1:
40.                 System.out.print("Masukkan No Rekening: ");
41.                 String no = input.nextLine();
42.                 System.out.print("Masukkan Nama Pemilik: ");
43.                 String nama= input.nextLine();
44.                 System.out.print("Masukkan Saldo Awal: ");
45.                 if (!input.hasNextDouble()) {
46.                     System.out.println("Error: Saldo awal
harus berupa angka! Pembuatan rekening dibatalkan.");
47.                     input.next();
48.                     break;
49.                 }
50.                 double saldo = input.nextDouble();
51.                 input.nextLine();
52.
53.                 //Intansiasi Object / Menjalankan
Constructor
54.                 akunAktif = new Rekening(no, nama, saldo);
55.
56.                 //simpan ke dalam arraylist
57.                 daftarRekening.add(akunAktif);
58.                 break;
59.
60.             case 2:
61.                 if (akunAktif == null) {
62.                     System.out.println("Error:
Mohon Maaf, Anda belum memiliki nomor rekening!");
63.                 } else {
64.                     System.out.print("Masukkan
Nominal Setor: ");
65.                     double setor =
input.nextDouble();
66.                     akunAktif.setorTunai(setor);
67.                 }
68.                 break;
69.

```

```

70.         case 3:
71.             if (akunAktif == null) {
72.                 System.out.println("Error:
Mohon Maaf, Anda belum memiliki nomor rekening!");
73.             } else {
74.                 System.out.print("Masukkan
Nominal Tarik: ");
75.                 double Tarik =
input.nextDouble();
76.                 akunAktif.tarikTunai(Tarik);
77.             }
78.             break;
79.
80.         case 4:
81.             if (akunAktif == null) {
82.                 System.out.println("Error: Anda
bekum membuka rekening");
83.             } else {
84.                 akunAktif.cekInformasi();
85.             }
86.             break;
87.
88.         case 5:
89.             if (daftarRekening.isEmpty()) {
90.                 System.out.println("Error:
Belum ada rekening yang terdaftar!");
91.             } else {
92.                 System.out.print("Masukkan
Nomor Rekening yang ingin diaktifkan: ");
93.                 String noCari =
input.nextLine();
94.
95.                 Rekening ditemukan = null;
96.                 for (Rekening r :
daftarRekening) {
97.                     if
(r.getNomorRekening().contentEquals(noCari)) {
98.                         ditemukan = r;
99.                         break;
100.                     }
101.                 }
102.                 if (ditemukan != null) {
103.                     akunAktif =
ditemukan;
104.
System.out.println("Sukses: Akun aktif berhasil diubah ke
no rekening " + akunAktif.getNomorRekening());
105.                 } else {
106.
System.out.println("Error: Nomor rekening tidak
ditemukan!");
107.                 }
108.             }
109.             break;

```

```

110.
111.         case 6:
112.             if (akunAktif == null) {
113.                 System.out.println("Error: Anda belum membuka rekening
114.                 ");
115.             } else {
116.                 System.out.println("===
117.                 MUTASI REKENING ===");
118.                 akunAktif.cetakMutasi();
119.             }
120.             break;
121.         case 7:
122.         case 8:
123.             System.out.println("Total
124.             Setor : " );
125.         case 0:
126.             isRunning = false;
127.             System.out.println("Sistem
128.             ditutup. Terima Kasih!");
129.             break;
130.         default:
131.             System.out.println("Pilihlah Tidak Valid");
132.         }
133.     }
134.     input.close();
135. }
136. }

```

Class Main merupakan class utama yang digunakan untuk menjalankan sistem perbankan mini dan mengatur interaksi dengan pengguna. Program menggunakan Scanner untuk menerima input, ArrayList<Rekening> untuk menyimpan beberapa objek rekening, serta akunAktif untuk menentukan rekening yang sedang digunakan. Program berjalan menggunakan perulangan while dan menyediakan menu untuk membuka rekening, setor tunai, tarik tunai, melihat informasi rekening, mengganti akun, mencetak mutasi, dan keluar dari program.

Pada saat memilih menu buka rekening, program membuat objek Rekening menggunakan constructor kemudian menyimpannya ke dalam daftarRekening. Menu setor dan tarik tunai akan memanggil method

setorTunai() atau tarikTunai() dari objek Rekening yang sedang aktif. Menu ganti akun digunakan untuk mencari rekening berdasarkan nomor rekening dengan melakukan perulangan pada daftarRekening, sedangkan menu cek informasi dan cetak mutasi masing-masing memanggil method cekInformasi () dan cetakMutasi () pada akunAktif.

Dengan demikian, class Main berperan sebagai pengendali jalannya program, sedangkan pengelolaan data rekening dilakukan oleh Rekening dan penyimpanan detail transaksi dilakukan oleh Transaksi. Hubungan ketiga class tersebut dapat dilihat sebagai Main → Rekening → banyak Transaksi, yang menunjukkan penerapan asosiasi one-to-many. Fitur pencatatan dan pencetakan riwayat transaksi ini merupakan inti pengembangan pada praktikum kedua.

2.2.3 Class Transaksi

Class selanjutnya yaitu class Transaksi yang merupakan class tambahan dari pekan sebelumnya. Kode program Transaksi digunakan untuk menyimpan data dari setiap transaksi yang dilakukan. Class ini mencatat ID transaksi, jenis transaksi, dan nominal transaksi. Class ini juga digunakan untuk menampilkan detail riwayat transaksi. Berikut adalah kode program dari class Transaksi:

```

1. package pekan2;
2.
3. public class Transaksi {
4.     String idTransaksi;
5.     String jenis;
6.     double nominal;
7.
8.     //constructor
9.     public Transaksi (String id, String jenis, double
    nominal) {
10.         this.idTransaksi = id;
11.         this.jenis = jenis;
12.         this.nominal = nominal;
13.     }
14.
15.     public void cetakDetail () {

```

```

16.         System.out.println("ID: " + idTransaksi + " |
    Jenis: " + jenis + " | Nominal : " + nominal);
17.     }
18.
19. }

```

Class Transaksi digunakan untuk merepresentasikan data dari setiap transaksi yang terjadi pada rekening. Class ini memiliki tiga atribut, yaitu idTransaksi untuk menyimpan identitas transaksi, jenis untuk menentukan jenis transaksi, dan nominal untuk menyimpan jumlah transaksi. Jenis transaksi yang digunakan adalah "Kredit" untuk penyetoran dan "Debit" untuk penarikan, sesuai dengan penjelasan pada modul praktikum.

Constructor Transaksi (String id, String jenis, double nominal) digunakan untuk memberikan nilai awal pada ketiga atribut ketika objek Transaksi dibuat. Dengan adanya constructor tersebut, setiap objek transaksi dapat memiliki ID, jenis, dan nominal masing-masing. Pada program, objek Transaksi dibuat ketika terjadi transaksi setor atau tarik tunai yang berhasil.

Class ini juga memiliki method cetakDetail () yang digunakan untuk menampilkan informasi setiap transaksi berupa ID, jenis, dan nominal. Method tersebut nantinya dipanggil oleh class Rekening melalui method cetakMutasi () untuk menampilkan seluruh riwayat transaksi. Hal ini sesuai dengan modul yang menjelaskan bahwa cetakDetail () digunakan untuk menampilkan satu baris detail transaksi.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa class diagram dapat digunakan sebagai gambaran struktur dan hubungan antar class sebelum diimplementasikan ke dalam program Java. Pada praktikum ini, relasi asosiasi *one-to-many* diterapkan antara class Rekening dan Transaksi, di mana satu rekening dapat memiliki banyak transaksi yang disimpan dalam ArrayList. Implementasi tersebut memungkinkan program mencatat transaksi setor dan tarik tunai serta menampilkan riwayat transaksi, sehingga konsep relasi antar objek dapat diterapkan secara langsung pada sistem perbankan mini.

3.2 Saran

Dalam pelaksanaan praktikum berikutnya, pemahaman terhadap class diagram dan hubungan antar objek perlu ditingkatkan agar proses penerjemahan diagram ke dalam kode Java dapat dilakukan dengan lebih baik. Praktikan juga sebaiknya memperhatikan struktur program, penggunaan atribut, method, constructor, serta ArrayList agar hubungan antar objek dapat diterapkan dengan tepat. Selain itu, program dapat dikembangkan dengan menambahkan fitur lain yang masih berkaitan dengan pengelolaan rekening dan transaksi.

DAFTAR PUSTAKA

[1] ITBOX, “Class Diagram: Pengertian, Simbol, Manfaat beserta Contohnya,”. [Daring]. Tersedia pada: [Class Diagram: Pengertian, Simbol, Manfaat beserta Contohnya](#)[Diakses: 18-September-2026].

[2] Dimas Dani, “Pahami Konsep Association, Aggregation, Composition pada Java,”. [Daring]. Tersedia pada: [Pahami Konsep Association, Aggregation, Composition pada Java | by Dimas Dani | Medium](#) [Diakses: 18-September-2026].

LAPORAN TUGAS PRAKTIKUM PEMOGRAMAN BERORIENTASI OBJEK

(Relasi Antar Objek (Class Diagram & Asosiasi))

Dosen Pengampu:

Nurfiah, S.ST, M.Kom.

Asisten Praktikum:

M Dawi Syauqi



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

Deskripsi Tugas

Tugas 1: Integrasi Rekam Jejak Penarikan

- Buka kembali method tarikTunai(double nominal) di dalam file Rekening.java.
- Tambahkan logika perekaman objek Transaksi seperti pada contoh setorTunai. Pastikan jenis transaksinya dicatat sebagai "Debit" dan kodenya menggunakan prefiks "TRX-T-". Transaksi hanya boleh dicatat jika penarikan berhasil (saldo mencukupi).

Tugas 2: Fitur Cetak Mutasi Rekening

- Buat sebuah method baru di dalam class Rekening.java dengan nama cetakMutasi().
- Di dalam method tersebut, gunakan struktur perulangan for-each untuk menelusuri seluruh data di dalam koleksi riwayatTransaksi. Panggil method cetakDetail() milik setiap objek Transaksi. Jika riwayat kosong, cetak pesan: "Belum ada transaksi pada rekening ini".
- Buka file Main.java, tambahkan menu baru: "6. Cetak Mutasi (Riwayat)". Panggil method cetakMutasi() milik akunAktif saat pengguna memilih menu tersebut.

PEMBAHASAN TUGAS

1. Integrasi Rekam Jejak Penarikan

Method `tarikTunai(double nominal)` digunakan untuk melakukan proses penarikan uang dari rekening sekaligus mencatat transaksi penarikan tersebut ke dalam riwayat transaksi. Method ini memiliki parameter nominal yang digunakan untuk menentukan jumlah uang yang akan ditarik. Berikut adalah kode program tugas 1:

```

1. public void tarikTunai(double nominal) {
2.     if (nominal < 10000) {
3.         System.out.println("Transaksi Gagal: Minimal nominal
   penarikan 10.000");
4.     }
5.     else if (nominal > saldo) {
6.         System.out.println("Transaksi Gagal: Saldo tidak
   mencukupi. Saldo Anda: Rp" + saldo);
7.     }
8.     else {
9.         // Merekam riwayat (pembuatan objek Transaksi di dalam
   method)
10.        String idTrx = "TRX-T-" + System.currentTimeMillis();
11.        Transaksi trxBaru = new Transaksi (idTrx, "Debit",
   nominal);
12.        riwayatTransaksi.add(trxBaru);
13.        saldo -= nominal;
14.        System.out.println("Tarik tunai Rp" + nominal + "
   berhasil. Saldo saat ini: Rp" + saldo);
15.    }
16. }

```

Kode program inti dari tugas satu dimulai dari `else` setelah memenuhi beberapa `if` terlebih dahulu yaitu minimal penarikan 10000 dan tidak lebih dari saldo. Jika kedua kondisi tersebut tidak terpenuhi, program masuk ke bagian `else`, yang merupakan bagian utama dari Tugas 1. Baris `String idTrx = "TRX-T-" + System.currentTimeMillis();` digunakan untuk membuat ID transaksi dengan awalan TRX-T-. Penggunaan awalan tersebut sesuai dengan ketentuan modul bahwa transaksi penarikan dicatat menggunakan prefiks TRX-T-.

Selanjutnya, baris `Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);` membuat objek baru dari class `Transaksi`. Objek tersebut menyimpan ID transaksi, jenis transaksi "Debit" yang menunjukkan penarikan, serta nominal uang yang ditarik.

Setelah objek transaksi dibuat, `riwayatTransaksi.add(trxBaru);` digunakan untuk memasukkan transaksi tersebut ke dalam koleksi `riwayatTransaksi` pada class `Rekening`. Setelah transaksi berhasil dicatat, `baris saldo -= nominal;` mengurangi saldo rekening sesuai dengan jumlah uang yang ditarik. Terakhir, `System.out.println(...)` menampilkan informasi bahwa penarikan berhasil beserta saldo rekening setelah transaksi dilakukan.

2. Fitur Cetak Mutasi Rekening

Method `cetakMutasi()` digunakan untuk menampilkan seluruh riwayat transaksi yang tersimpan pada suatu rekening. Method ini memanfaatkan koleksi `riwayatTransaksi` yang berisi objek-objek `Transaksi`. Sesuai dengan instruksi modul, data tersebut ditelusuri menggunakan perulangan `for-each` dan setiap transaksi ditampilkan menggunakan method `cetakDetail()`. Jika belum terdapat transaksi, program menampilkan pesan bahwa belum ada transaksi pada rekening tersebut. Kode program tugas 2 sebagai berikut:

- Class Rekening

```

1. public void cetakMutasi() {
2.     if (riwayatTransaksi.isEmpty()) {
3.         System.out.println("Belum ada transaksi pada
   rekening ini");
4.     } else {
5.         for (Transaksi t : riwayatTransaksi) {
6.             t.cetakDetail();
7.         }
8.     }
9. }
```

Pada bagian `if (riwayatTransaksi.isEmpty())`, program memeriksa apakah koleksi `riwayatTransaksi` masih kosong. Jika kosong, program menjalankan perintah:

```
System.out.println("Belum ada transaksi pada rekening ini");
```

Pesan tersebut menunjukkan bahwa rekening belum memiliki transaksi yang tersimpan. Jika riwayat transaksi tidak kosong, program masuk ke bagian else. Selanjutnya, digunakan perulangan for-each. Perulangan tersebut digunakan untuk mengambil setiap objek Transaksi yang terdapat di dalam riwayatTransaksi. Variabel t digunakan sebagai objek Transaksi yang sedang diproses. Kemudian, t.cetakDetail() dipanggil untuk menampilkan detail dari setiap transaksi.

- Class Main

Selanjutnya adalah menambahkan menu 6. Cetak Mutasi (Riwayat) pada Main.java, kemudian memanggil cetakMutasi() ketika pengguna memilih menu tersebut.

```
System.out.println("6. Cetak Mutasi (Riwayat)");
```

```
1. case 6:
2.         if (akunAktif == null) {
3.             System.out.println("Error: Anda belum
   membuka rekening ");
4.         } else {
5.             System.out.println("=== MUTASI
   REKENING ===");
6.             akunAktif.cetakMutasi();
7.         }
8.         break;
```

ketika pengguna memilih menu nomor 6, program menjalankan akunAktif.cetakMutasi() dengan syarat if akunAktif tidak null. Artinya, method cetakMutasi() pada rekening yang sedang aktif akan dipanggil sehingga seluruh riwayat transaksi rekening dapat ditampilkan.

3. Output Kode Program Tugas

Berikut adalah beberapa output yang relevan dengan tugas 2:

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Challenge 3
0. Keluar
Pilih Menu:
3
Masukkan Nominal Tarik: 100000
Tarik tunai Rp100000.0 berhasil. Saldo saat ini: Rp900000.0

```

```

==== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Challenge 3
0. Keluar
Pilih Menu:
1
Masukkan No Rekening: 221006
Masukkan Nama Pemilik: Sofian
Masukkan Saldo Awal: 1000000
Rekening atas nama Sofian berhasil dibuat dengan saldo Rp1000000.0

```

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Challenge 3
0. Keluar
Pilih Menu:
2
Masukkan Nominal Setor: 600000
Gagal: Nominal setor harus lebih dari 0! atau Transaksi lebih dari 500000

```

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Challenge 3
0. Keluar
Pilih Menu:
6
|==== MUTASI REKENING ====
ID: TRX-T-1789913722231 | Jenis: Debit | Nominal :100000.0

```